

EJERCICIO 1

Hay 3 formas de automatizar tareas

Triggers: Conjunto de instrucciones que se ejecutan ante un posible evento, por ejemplo, un BEFORE INSERT o AFTER DELETE

Procedimientos: Bloques de código que se guardan en el servidor para ser reutilizados

Funciones: Similares a los procedimientos, solo que estos están hechos para retornar un dato

A diferencia de procedimiento y funciones, el procedimiento se llama con CALL y la función con SELECT

EJERCICIO 2

Hay 3 tipos de métodos

Mediante herramientas de línea de comandos (CLI), que es directa y muy usado para automatización

Mediante entornos de desarrollo IDE (GUI): Son herramientas visuales, como MySQL Workbench, PgAdmin entre otros

Mediante ejecución de scripts: Se usa SQL en sentencia desde un lenguaje de programación (Java, Python...)

EJERCICIO 3

Mediante editor de texto plano (Básico): Como un notepad

Mediante editor de texto enriquecido (Genérico): Que tiene funciones adicionales (Notepad++)

Mediante un IDE: Como VSCode, NetBeans

Mediante una línea de comandos con funciones de IDE: Es una CLI pero que mediante plugins/Extensiones puede invocar herramientas de ejecución

Ejercicio 4

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ContarProductosPorLinea`(  
    IN p_productLine VARCHAR(50),  
    OUT p_total INT  
)  
  
BEGIN  
    SELECT COUNT(*)  
    INTO p_total  
    FROM products  
    WHERE productLine = p_productLine;  
  
END
```

Ejercicio 5

```
CREATE DEFINER=`root`@`localhost` PROCEDURE `ResumenVentasCliente`(  
    IN p_customerNumber INT,  
    OUT p_nombre VARCHAR(50),  
    OUT p_cantidadPagos DECIMAL(10,2),  
    OUT p_sumaPagos DECIMAL(10,2),  
    OUT p_promedioPagos DECIMAL(10,2)  
)  
  
BEGIN  
    IF p_customerNumber IS NOT NULL and p_customerNumber > 0 THEN  
  
        SELECT customerName INTO p_nombre  
        FROM customers  
        WHERE customerNumber = p_customerNumber;  
  
        SELECT COUNT(*) INTO p_cantidadPagos
```

```

FROM payments

WHERE customerNumber = p_customerNumber;

SELECT SUM(amount) INTO p_sumaPagos

FROM payments

WHERE customerNumber = p_customerNumber;

SELECT AVG(amount) INTO p_promedioPagos

FROM payments

WHERE customerNumber = p_customerNumber;

ELSE

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'Error, el cliente no existe en la tabla';

END IF;

END

```

EJERCICIO 6

```

CREATE DEFINER=`root`@`localhost` FUNCTION `EvaluarPrecioProducto`(p_productCode
VARCHAR(50)

) RETURNS varchar(50) CHARSET utf8mb4 COLLATE utf8mb4_general_ci

BEGIN

    DECLARE precioCompraPromedio DECIMAL(10,2);

    DECLARE mensaje VARCHAR(50);

    DECLARE precioCompraNormal DECIMAL(10,2);

    SELECT buyPrice INTO precioCompraNormal

    FROM products

    WHERE productCode = p_productCode;

```

```
SELECT AVG(buyPrice) into precioCompraPromedio  
  
FROM products  
  
WHERE productCode = p_productCode AND productCode = (
```

```
SELECT productLine  
  
FROM products  
  
WHERE p_productCode = productCode);
```

```
IF precioCompraNormal > precioCompraPromedio THEN  
  
    SET mensaje ='Precio superior al promedio';  
  
ELSEIF precioCompraNormal < precioCompraPromedio THEN  
  
    SET mensaje ='Precio inferior al promedio';  
  
ELSE SET mensaje = 'Precio exacto al promedio';  
  
END IF;
```

```
RETURN mensaje;
```

```
END
```

EJERCICIO 7

```
CREATE DEFINER='root'@'localhost' TRIGGER `classicmodels`.`tg_validar_pago_antes_insert`  
BEFORE INSERT ON `payments` FOR EACH ROW
```

```
BEGIN
```

```
IF new.amount < 0 OR new.paymentDate <> CURDATE() THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT ='El monto no puede ser 0 o inferior y la fecha no puede ser distinta a  
la actual';
```

```
ELSEIF new.amount < 0 AND new.paymentDate = CURDATE() THEN
```

```

        SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT ='El monto no puede ser 0';

        ELSEIF new.amount > 0 AND new.paymentDate <> CURDATE() THEN

        SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT ='La fecha no puede ser distinta a la actual';

        ELSE

INSERT INTO payments(customerNumber, checkNumber, paymentDate, amount)

VALUES (new.customerNumber, new.checkNumber, new.paymentDate, new.amount);

END IF;

END

```

EJERCICIO 8

Comentario 1: Empezamos el planificador de eventos

Comentario 2: Creamos el evento “evt_archivar_pedidos_simples” para que se ejecute a diario

Comentario 3: Declaramos la salida del handler para la SQLException

Comentario 4: Lógica de actualización → Actualizamos orders y le ponemos el estado a “Archivado” cuando la fecha de pedido haya superado 1 año de antigüedad y que además esté enviado

EJERCICIO 9

Comentario 1: Declaramos las variables

Comentario 2: Declaramos el cursos y su select

Comentario 3: Declaramos el handler para cuando termine

Comentario 4: Abrimos el cursos

Comentario 5: Abrimos el bucle de lectura

Comentario 6: Lógica de actualización: Si el crédito actual es superior al umbral, entonces añadimos el crédito actual a la suma

Comentario 7: Cerramos el cursos